Chapter 8. Grouping and Aggregates

Data is generally stored at the lowest level of granularity needed by any of a database's users; if Chuck in accounting needs to look at individual customer transactions, then there needs to be a table in the database that stores individual transactions. That doesn't mean, however, that all users must deal with the data as it is stored in the database. The focus of this chapter is on how data can be grouped and aggregated to allow users to interact with it at some higher level of granularity than what is stored in the database.

Grouping Concepts

Sometimes you will want to find trends in your data that will require the database server to cook the data a bit before you can generate the results you are looking for. For example, let's say that you are in charge of sending coupons for free rentals to your best customers. You could issue a simple query to look at the raw data:

```
mysql> SELECT customer_id FROM rental;
+-------------+
| customer_id |
+-------------+
|           1 |
|           1 |
|           1 |
|           1 |
|           1 |
|           1 |
|           1 |
...
|         599 |
|         599 |
|         599 |
|         599 |
|         599 |
|         599 |
```

```
+-------------+
16044 rows in set (0.01 sec)
```

With 599 customers spanning more than 16,000 rental records, it isn't feasible to determine which customers have rented the most films by looking at the raw data. Instead, you can ask the database server to group the data for you by using the `group by` clause. Here's the same query but employing a `group by` clause to group the rental data by customer ID:

```
mysql> SELECT customer_id
    -> FROM rental
    -> GROUP BY customer_id;
+-------------+
| customer_id |
+-------------+
|           1 |
|           2 |
|           3 |
|           4 |
|           5 |
|           6 |
...
|         594 |
|         595 |
|         596 |
|         597 |
|         598 |
|         599 |
+-------------+
599 rows in set (0.00 sec)
```

The result set contains one row for each distinct value in the `customer_id` column, resulting in 599 rows instead of the full 16,044 rows. The reason for the smaller result set is that some of the customers rented more than one film. To see how many films each customer rented, you can use an *aggregate function* in the `select` clause to count the number of rows in each group:

```
mysql> SELECT customer_id, count(*)
    -> FROM rental
    -> GROUP BY customer_id;
+-------------+----------+
| customer_id | count(*) |
```

```
+-------------+----------+
|         1 |      32 |
|         2 |      27 |
|         3 |      26 |
|         4 |      22 |
|         5 |      38 |
|         6 |      28 |
...
|       594 |      27 |
|       595 |      30 |
|       596 |      28 |
|       597 |      25 |
|       598 |      22 |
|       599 |      19 |
+-------------+----------+
599 rows in set (0.01 sec)
```

The aggregate function `count()` counts the number of rows in each group, and the asterisk tells the server to count everything in the group. Using the combination of a `group by` clause and the `count()` aggregate function, you are able to generate exactly the data needed to answer the business question without having to look at the raw data.

Looking at the results, you can see that 32 films were rented by customer ID 1, and 25 films were rented by the customer ID 597. In order to determine which customers have rented the most films, simply add an `order by` clause:

```
mysql> SELECT customer_id, count(*)
    -> FROM rental
    -> GROUP BY customer_id
    -> ORDER BY 2 DESC;
+-------------+----------+
| customer_id | count(*) |
+-------------+----------+
|       148 |      46 |
|       526 |      45 |
|       236 |      42 |
|       144 |      42 |
|        75 |      41 |
...
|       248 |      15 |
|       110 |      14 |
|       281 |      14 |
|        61 |      14 |
|       318 |      12 |
```

```
+-------------+----------+
599 rows in set (0.01 sec)
```

Now that the results are sorted, you can easily see that customer ID 148 has rented the most films (46), while customer ID 318 has rented the fewest films (12).

When grouping data, you may need to filter out undesired data from your result set based on groups of data rather than based on the raw data. Since the `group by` clause runs *after* the `where` clause has been evaluated, you cannot add filter conditions to your `where` clause for this purpose. For example, here's an attempt to filter out any customers who have rented fewer than 40 films:

```
mysql> SELECT customer_id, count(*)
    -> FROM rental
    -> WHERE count(*) >= 40
    -> GROUP BY customer_id;
ERROR 1111 (HY000): Invalid use of group function
```

You cannot refer to the aggregate function `count(*)` in your `where` clause, because the groups have not yet been generated at the time the `where` clause is evaluated. Instead, you must put your group filter conditions in the `having` clause. Here's what the query would look like using `having`:

```
mysql> SELECT customer_id, count(*)
    -> FROM rental
    -> GROUP BY customer_id
    -> HAVING count(*) >= 40;
+-------------+----------+
| customer_id | count(*) |
+-------------+----------+
|          75 |       41 |
|         144 |       42 |
|         148 |       46 |
|         197 |       40 |
|         236 |       42 |
|         469 |       40 |
|         526 |       45 |
+-------------+----------+
7 rows in set (0.01 sec)
```

Because those groups containing fewer than 40 members have been filtered out via the `having` clause, the result set now contains only those customers who have rented 40 or more films.

Aggregate Functions

Aggregate functions perform a specific operation over all rows in a group. Although every database server has its own set of specialty aggregate functions, the common aggregate functions implemented by all major servers include:

`max()`
: Returns the maximum value within a set

`min()`
: Returns the minimum value within a set

`avg()`
: Returns the average value across a set

`sum()`
: Returns the sum of the values across a set

`count()`
: Returns the number of values in a set

Here's a query that uses all of the common aggregate functions to analyze the data on film rental payments:

```
mysql> SELECT MAX(amount) max_amt,
    ->   MIN(amount) min_amt,
    ->   AVG(amount) avg_amt,
    ->   SUM(amount) tot_amt,
    ->   COUNT(*) num_payments
    -> FROM payment;
+---------+---------+----------+----------+--------------+
| max_amt | min_amt | avg_amt  | tot_amt  | num_payments |
+---------+---------+----------+----------+--------------+
|   11.99 |    0.00 | 4.200667 | 67416.51 |        16049 |
```

```
+---------+---------+----------+----------+-------------+
1 row in set (0.09 sec)
```

The results from this query tell you that, across the 16,049 rows in the `payment` table, the maximum amount paid to rent a film was $11.99, the minimum amount was $0, the average payment was $4.20, and the total of all rental payments was $67,416.51. Hopefully, this gives you an appreciation for the role of these aggregate functions; the next subsections further clarify how you can utilize these functions.

Implicit Versus Explicit Groups

In the previous example, every value returned by the query is generated by an aggregate function. Since there is no `group by` clause, there is a single, *implicit* group (all rows in the `payment` table).

In most cases, however, you will want to retrieve additional columns along with columns generated by aggregate functions. What if, for example, you wanted to extend the previous query to execute the same five aggregate functions for *each* customer, instead of across all customers? For this query, you would want to retrieve the `customer_id` column along with the five aggregate functions, as in:

```
SELECT customer_id,
  MAX(amount) max_amt,
  MIN(amount) min_amt,
  AVG(amount) avg_amt,
  SUM(amount) tot_amt,
  COUNT(*) num_payments
FROM payment;
```

However, if you try to execute the query, you will receive the following error:

```
ERROR 1140 (42000): In aggregated query without GROUP BY,
  expression #1 of SELECT list contains nonaggregated column
```

While it may be obvious to you that you want the aggregate functions applied to each customer found in the `payment` table, this query fails because you have not *explicitly* specified how the data should be grouped. Therefore, you will need to add a `group by` clause to specify over which group of rows the aggregate functions should be applied:

```
mysql> SELECT customer_id,
    ->   MAX(amount) max_amt,
    ->   MIN(amount) min_amt,
    ->   AVG(amount) avg_amt,
    ->   SUM(amount) tot_amt,
    ->   COUNT(*) num_payments
    -> FROM payment
    -> GROUP BY customer_id;
+-------------+---------+---------+----------+---------+--------------+
| customer_id | max_amt | min_amt | avg_amt  | tot_amt | num_payments |
+-------------+---------+---------+----------+---------+--------------+
|           1 |    9.99 |    0.99 | 3.708750 |  118.68 |           32 |
|           2 |   10.99 |    0.99 | 4.767778 |  128.73 |           27 |
|           3 |   10.99 |    0.99 | 5.220769 |  135.74 |           26 |
|           4 |    8.99 |    0.99 | 3.717273 |   81.78 |           22 |
|           5 |    9.99 |    0.99 | 3.805789 |  144.62 |           38 |
|           6 |    7.99 |    0.99 | 3.347143 |   93.72 |           28 |
...
|         594 |    8.99 |    0.99 | 4.841852 |  130.73 |           27 |
|         595 |   10.99 |    0.99 | 3.923333 |  117.70 |           30 |
|         596 |    6.99 |    0.99 | 3.454286 |   96.72 |           28 |
|         597 |    8.99 |    0.99 | 3.990000 |   99.75 |           25 |
|         598 |    7.99 |    0.99 | 3.808182 |   83.78 |           22 |
|         599 |    9.99 |    0.99 | 4.411053 |   83.81 |           19 |
+-------------+---------+---------+----------+---------+--------------+
599 rows in set (0.04 sec)
```

With the inclusion of the `group by` clause, the server knows to group together rows having the same value in the `customer_id` column first and then to apply the five aggregate functions to each of the 599 groups.

Counting Distinct Values

When using the `count()` function to determine the number of members in each group, you have your choice of counting *all* members in the group or counting only the *distinct* values for a column across all members of the group.

For example, consider the following query, which uses the `count()` function with the `customer_id` column in two different ways:

```
mysql> SELECT COUNT(customer_id) num_rows,
    ->   COUNT(DISTINCT customer_id) num_customers
```

```
    -> FROM payment;
+----------+---------------+
| num_rows | num_customers |
+----------+---------------+
|    16049 |           599 |
+----------+---------------+
1 row in set (0.01 sec)
```

The first column in the query simply counts the number of rows in the `payment` table, whereas the second column examines the values in the `customer_id` column and counts only the number of unique values. By specifying `distinct`, therefore, the `count()` function examines the values of a column for each member of the group in order to find and remove duplicates, rather than simply counting the number of values in the group.

Using Expressions

Along with using columns as arguments to aggregate functions, you can use expressions as well. For example, you may want to find the maximum number of days between when a film was rented and subsequently returned. You can achieve this via the following query:

```
mysql> SELECT MAX(datediff(return_date,rental_date))
    -> FROM rental;
+---------------------------------------+
| MAX(datediff(return_date,rental_date)) |
+---------------------------------------+
|                                    10 |
+---------------------------------------+
1 row in set (0.01 sec)
```

The `datediff` function is used to compute the number of days between the return date and the rental date for every rental, and the `max` function returns the highest value, which in this case is 10 days.

While this example uses a fairly simple expression, expressions used as arguments to aggregate functions can be as complex as needed, as long as they return a number, string, or date. In Chapter 11, I show you how you can use `case` expressions with aggregate functions to determine whether a particular row should or should not be included in an aggregation.

How Nulls Are Handled

When performing aggregations, or, indeed, any type of numeric calculation, you should always consider how `null` values might affect the outcome of your calculation. To illustrate, I will build a simple table to hold numeric data and populate it with the set {1, 3, 5}:

```
mysql> CREATE TABLE number_tbl
    ->  (val SMALLINT);
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO number_tbl VALUES (1);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO number_tbl VALUES (3);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO number_tbl VALUES (5);
Query OK, 1 row affected (0.00 sec)
```

Consider the following query, which performs five aggregate functions on the set of numbers:

```
mysql> SELECT COUNT(*) num_rows,
    ->   COUNT(val) num_vals,
    ->   SUM(val) total,
    ->   MAX(val) max_val,
    ->   AVG(val) avg_val
    -> FROM number_tbl;
+----------+----------+-------+---------+---------+
| num_rows | num_vals | total | max_val | avg_val |
+----------+----------+-------+---------+---------+
|        3 |        3 |     9 |       5 |  3.0000 |
+----------+----------+-------+---------+---------+
1 row in set (0.08 sec)
```

The results are as you would expect: both `count(*)` and `count(val)` return the value `3`, `sum(val)` returns the value `9`, `max(val)` returns `5`, and `avg(val)` returns `3`. Next, I will add a `null` value to the `number_tbl` table and run the query again:

```
mysql> INSERT INTO number_tbl VALUES (NULL);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT COUNT(*) num_rows,
    ->   COUNT(val) num_vals,
    ->   SUM(val) total,
    ->   MAX(val) max_val,
    ->   AVG(val) avg_val
    -> FROM number_tbl;
+----------+----------+-------+---------+---------+
| num_rows | num_vals | total | max_val | avg_val |
+----------+----------+-------+---------+---------+
|        4 |        3 |     9 |       5 |  3.0000 |
+----------+----------+-------+---------+---------+
1 row in set (0.00 sec)
```

Even with the addition of the `null` value to the table, the `sum()`, `max()`, and `avg()` functions all return the same values, indicating that they ignore any `null` values encountered. The `count(*)` function now returns the value `4`, which is valid since the `number_tbl` table contains four rows, while the `count(val)` function still returns the value `3`. The difference is that `count(*)` counts the number of rows, whereas `count(val)` counts the number of *values* contained in the `val` column and ignores any `null` values encountered.

Generating Groups

People are rarely interested in looking at raw data; instead, people engaging in data analysis will want to manipulate the raw data to better suit their needs. Examples of common data manipulations include:

- Generating totals for a geographic region, such as total European sales
- Finding outliers, such as the top salesperson for 2020
- Determining frequencies, such as the number of films rented in each month

To answer these types of queries, you will need to ask the database server to group rows together by one or more columns or expressions. As you have seen already in several examples, the `group by` clause is the mechanism for grouping data within a query. In this section, you will see how to group data by one or more columns, how to group data using expressions, and how to generate rollups within groups.

Single-Column Grouping

Single-column groups are the simplest and most-often-used type of grouping. If you want to find the number of films associated with each actor, for example, you need only group on the `film_actor.actor_id` column, as in:

```
mysql> SELECT actor_id, count(*)
    -> FROM film_actor
    -> GROUP BY actor_id;
+----------+----------+
| actor_id | count(*) |
+----------+----------+
|        1 |       19 |
|        2 |       25 |
|        3 |       22 |
|        4 |       22 |
...
|      197 |       33 |
|      198 |       40 |
|      199 |       15 |
|      200 |       20 |
+----------+----------+
200 rows in set (0.11 sec)
```

This query generates 200 groups, one for each actor, and then sums the number of films for each member of the group.

Multicolumn Grouping

In some cases, you may want to generate groups that span *more* than one column. Expanding on the previous example, imagine that you want to find the total number of films for each film rating (G, PG, ...) for each actor. The following example shows how you can accomplish this:

```
mysql> SELECT fa.actor_id, f.rating, count(*)
    -> FROM film_actor fa
    ->   INNER JOIN film f
    ->   ON fa.film_id = f.film_id
    -> GROUP BY fa.actor_id, f.rating
    -> ORDER BY 1,2;
+----------+--------+----------+
| actor_id | rating | count(*) |
```

```
+----------+--------+----------+
|      1 | G      |       4 |
|      1 | PG     |       6 |
|      1 | PG-13  |       1 |
|      1 | R      |       3 |
|      1 | NC-17  |       5 |
|      2 | G      |       7 |
|      2 | PG     |       6 |
|      2 | PG-13  |       2 |
|      2 | R      |       2 |
|      2 | NC-17  |       8 |
...
|    199 | G      |       3 |
|    199 | PG     |       4 |
|    199 | PG-13  |       4 |
|    199 | R      |       2 |
|    199 | NC-17  |       2 |
|    200 | G      |       5 |
|    200 | PG     |       3 |
|    200 | PG-13  |       2 |
|    200 | R      |       6 |
|    200 | NC-17  |       4 |
+----------+--------+----------+
996 rows in set (0.01 sec)
```

This version of the query generates 996 groups, one for each combination of actor and film rating found by joining the `film_actor` table with the `film` table. Along with adding the `rating` column to the `select` clause, I also added it to the `group by` clause, since `rating` is retrieved from a table and is not generated via an aggregate function such as `max` or `count`.

Grouping via Expressions

Along with using columns to group data, you can build groups based on the values generated by expressions. Consider the following query, which groups rentals by year:

```
mysql> SELECT extract(YEAR FROM rental_date) year,
    ->   COUNT(*) how_many
    -> FROM rental
    -> GROUP BY extract(YEAR FROM rental_date);
+------+----------+
| year | how_many |
+------+----------+
```

```
| 2005 |    15862 |
| 2006 |      182 |
+------+----------+
2 rows in set (0.01 sec)
```

This query employs a fairly simple expression that uses the `extract()` function to return only the year portion of a date to group the rows in the `rental` table.

Generating Rollups

In "Multicolumn Grouping", I showed an example that counts the number of films for each actor and film rating. Let's say, however, that along with the total count for each actor/rating combination, you also want total counts for each distinct actor. You could run an additional query and merge the results, you could load the results of the query into a spreadsheet, or you could build a Python script, Java program, or some other mechanism to take that data and perform the additional calculations. Better yet, you could use the `with rollup` option to have the database server do the work for you. Here's the revised query using `with rollup` in the `group by` clause:

```
mysql> SELECT fa.actor_id, f.rating, count(*)
    -> FROM film_actor fa
    ->   INNER JOIN film f
    ->   ON fa.film_id = f.film_id
    -> GROUP BY fa.actor_id, f.rating WITH ROLLUP
    -> ORDER BY 1,2;
+----------+--------+----------+
| actor_id | rating | count(*) |
+----------+--------+----------+
|     NULL | NULL   |     5462 |
|        1 | NULL   |       19 |
|        1 | G      |        4 |
|        1 | PG     |        6 |
|        1 | PG-13  |        1 |
|        1 | R      |        3 |
|        1 | NC-17  |        5 |
|        2 | NULL   |       25 |
|        2 | G      |        7 |
|        2 | PG     |        6 |
|        2 | PG-13  |        2 |
|        2 | R      |        2 |
|        2 | NC-17  |        8 |
...
|      199 | NULL   |       15 |
```

```
|     199 | G     |       3 |
|     199 | PG    |       4 |
|     199 | PG-13 |       4 |
|     199 | R     |       2 |
|     199 | NC-17 |       2 |
|     200 | NULL  |      20 |
|     200 | G     |       5 |
|     200 | PG    |       3 |
|     200 | PG-13 |       2 |
|     200 | R     |       6 |
|     200 | NC-17 |       4 |
+----------+--------+----------+
1197 rows in set (0.07 sec)
```

There are now 201 additional rows in the result set, one for each of the 200 distinct actors and one for the grand total (all actors combined). For the 200 actor rollups, a `null` value is provided for the `rating` column, since the rollup is being performed across all ratings. Looking at the first line for `actor_id 200`, for example, you will see that a total of 20 films are associated with the actor; this equals the sum of the counts for each rating (4 NC-17 + 6 R + 2 PG-13 + 3 PG + 5 G). For the grand total row in the first line of the output, a `null` value is provided for both the `actor_id` and `rating` columns; the total for the first line of output equals 5,462, which is equal to the number of rows in the `film_actor` table.

NOTE

If you are using Oracle Database, you need to use a slightly different syntax to indicate that you want a rollup performed. The `group by` clause for the previous query would look as follows when using Oracle:

```
GROUP BY ROLLUP(fa.actor_id, f.rating)
```

The advantage of this syntax is that it allows you to perform rollups on a subset of the columns in the `group_by` clause. If you are grouping by columns a, b, and c, for example, you could indicate that the server should perform rollups on only columns b and c via the following:

```
GROUP BY a, ROLLUP(b, c)
```

If in addition to totals by actor you also want to calculate totals per rating, then you can use the `with cube` option, which will generate summary rows for *all* possible combinations of the grouping columns. Unfortunately, `with cube` is not available in version 8.0 of MySQL, but it is available with SQL Server and Oracle Database.

Group Filter Conditions

In Chapter 4, I introduced you to various types of filter conditions and showed how you can use them in the `where` clause. When grouping data, you also can apply filter conditions to the data *after* the groups have been generated. The `having` clause is where you should place these types of filter conditions. Consider the following example:

```
mysql> SELECT fa.actor_id, f.rating, count(*)
    -> FROM film_actor fa
    ->   INNER JOIN film f
    ->   ON fa.film_id = f.film_id
    -> WHERE f.rating IN ('G','PG')
    -> GROUP BY fa.actor_id, f.rating
    -> HAVING count(*) > 9;
+----------+--------+----------+
| actor_id | rating | count(*) |
+----------+--------+----------+
|      137 | PG     |       10 |
|       37 | PG     |       12 |
|      180 | PG     |       12 |
|        7 | G      |       10 |
|       83 | G      |       14 |
|      129 | G      |       12 |
|      111 | PG     |       15 |
|       44 | PG     |       12 |
|       26 | PG     |       11 |
|       92 | PG     |       12 |
|       17 | G      |       12 |
|      158 | PG     |       10 |
|      147 | PG     |       10 |
|       14 | G      |       10 |
|      102 | PG     |       11 |
|      133 | PG     |       10 |
+----------+--------+----------+
16 rows in set (0.01 sec)
```

This query has two filter conditions: one in the `where` clause, which filters out any films rated something other than G or PG, and another in the `having` clause, which filters out any actors who appeared in less than 10 films. Thus, one of the filters acts on data *before* it is grouped, and the other filter acts on data *after* the groups have been created. If you mistakenly put both filters in the `where` clause, you will see the following error:

```
mysql> SELECT fa.actor_id, f.rating, count(*)
    -> FROM film_actor fa
    ->   INNER JOIN film f
    ->   ON fa.film_id = f.film_id
    -> WHERE f.rating IN ('G','PG')
    ->   AND count(*) > 9
    -> GROUP BY fa.actor_id, f.rating;
ERROR 1111 (HY000): Invalid use of group function
```

This query fails because you cannot include an aggregate function in a query's `where` clause. This is because the filters in the `where` clause are evaluated *before* the grouping occurs, so the server can't yet perform any functions on groups.

WARNING

When adding filters to a query that includes a `group by` clause, think carefully about whether the filter acts on raw data, in which case it belongs in the `where` clause, or on grouped data, in which case it belongs in the `having` clause.

Test Your Knowledge

Work through the following exercises to test your grasp of SQL's grouping and aggregating features. Check your work with the answers in Appendix B.

Exercise 8-1

Construct a query that counts the number of rows in the `payment` table.

Exercise 8-2

Modify your query from Exercise 8-1 to count the number of payments made by each customer. Show the customer ID and the total amount paid for each customer.

Exercise 8-3

Modify your query from Exercise 8-2 to include only those customers who have made at least 40 payments.

Previous chapter

< 7. Data Generation, Manipulation, and Conversion

Next chapter

9. Subqueries >